FLUXTON

Senior Embedded Engineer

Take-Home Assignment

Sensor Gateway Firmware, simulated on your own computer

48 hours to submit · About 8–10 hours of work · No hardware needed

Submit as a git repository · Followed by a 60-minute review meeting

1. Overview

Thank you for taking the time to do this assignment. It is close to the work we do: firmware that has to keep running and keep data safe when its input is messy and its power is unreliable. There is no board to set up. You will build a small hardware layer that simulates the peripherals on a normal computer, and write the firmware on top of it.

Time to submit	48 hours from when you receive this brief
Expected effort	About 8–10 focused hours. Finish the core requirements first.
Language	Your choice (see section 7)
Hardware	None. Everything runs on Linux, macOS or Windows (WSL is fine)
Submission	A link to a git repository
Afterwards	A 60-minute review meeting where you walk us through your work

> **We are not looking for a perfect, finished product.** We want to see how you plan, how you reason about concurrency and failure, and how honestly you describe what you built and what you would do next.

2. The system

You are writing firmware for a small gateway. A sensor sends readings over UART. The gateway checks each frame, stores every reading in flash, and must not lose stored readings when power fails at any moment, including in the middle of a flash write. A debug console lets an engineer inspect the device.

The diagram shows only the outside of the system. How you structure the inside is part of what we assess, and you will describe it in your design document (section 6).

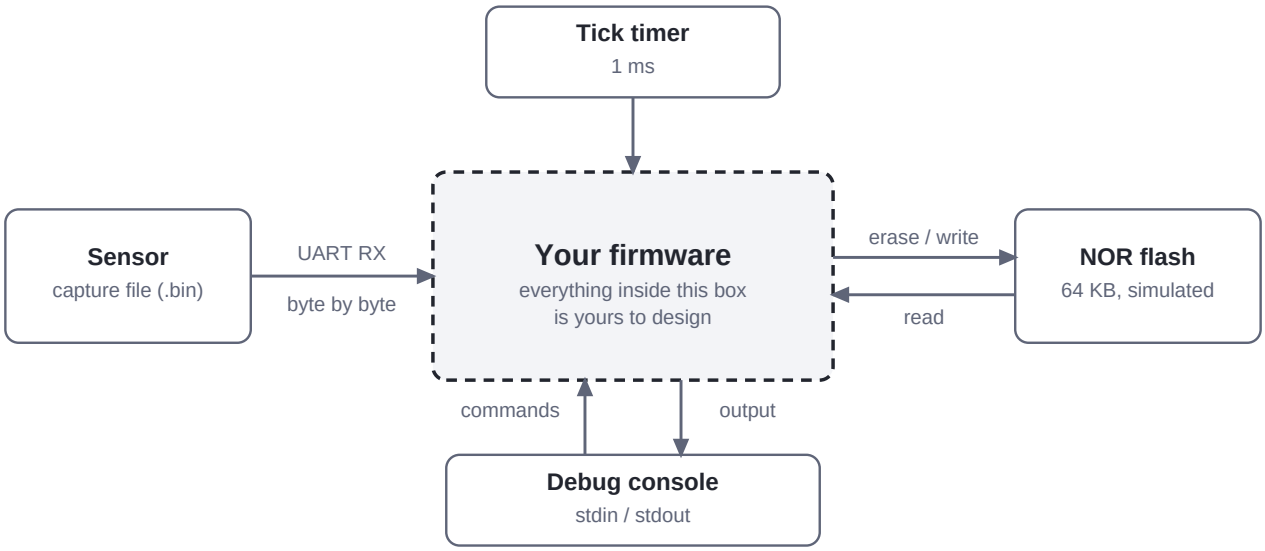


Figure 1. What the firmware talks to. All four peripherals are simulated by code you write.

3. What you receive

File	What it is
`This PDF`	The brief you are reading
`gen_frames.py`	Python 3 script that generates UART captures (standard library only). Use it to make extra test data.
`samples/clean.bin`	Valid frames only, intended for 115200 baud
`samples/noisy.bin`	Valid frames mixed with noise, corrupted, truncated and bad-length frames, intended for 115200 baud
`samples/burst.bin`	A long, dense stream with light noise, intended for 921600 baud
`samples/*.json`	Manifest for each capture: size, and how many intact frames it holds

A capture file is the raw byte stream exactly as it would arrive on the UART RX line, with no timing information. Your simulation decides when each byte arrives (section 5).

4. UART frame format

All multi-byte fields are little-endian.

Field	Size (bytes)	Description
SOF	1	Start of frame, always `0xAA`
LEN	1	Payload length, 0 to 64. A value above 64 is a length error.
TYPE	1	`0x01` reading, `0x02` heartbeat
PAYLOAD	LEN	Reading (LEN = 6): `int16` temperature in hundredths of °C, then `uint32` sensor timestamp in ms. Heartbeat: LEN = 0, no payload.
CRC	2	CRC-16/CCITT-FALSE over LEN, TYPE and PAYLOAD (not SOF)

CRC parameters: polynomial 0x1021, initial value 0xFFFF, no input or output reflection, final XOR 0x0000. Check value: the ASCII string “123456789” gives 0x29B1.

Frame rules

- A frame with a valid CRC but an unknown TYPE, or the wrong LEN for its TYPE, is counted as an unknown frame and ignored.
- Noise can contain `0xAA` bytes. A damaged or truncated frame can be followed immediately by a good one, and the good one may begin inside the bytes you were collecting for the damaged one. **Every intact frame in a capture must be found.** The manifests tell you how many there are.

Example frames

Meaning	Bytes on the wire (hex)
Reading: 23.45 °C at t = 1000 ms	`AA 06 01 29 09 E8 03 00 00 7B 84`
Reading: −5.12 °C at t = 70000 ms	`AA 06 01 00 FE 70 11 01 00 7D FC`
Heartbeat	`AA 00 02 4D 3D`

5. Core requirements

All five parts are required. Keep the firmware logic separate from the simulation, as if the same core will later run on a microcontroller.

5.1 Hardware layer

- Define interfaces for UART receive, a 1 ms tick, flash and the debug console. Firmware logic reaches hardware only through these interfaces.
- Implement them for your computer. A separate thread (or your language's equivalent) plays the role of the UART receive interrupt: it reads a capture file and delivers bytes one at a time at a configurable baud rate, using 10 bits per byte (8N1).

5.2 Interrupt-to-main-loop handoff

- Bytes delivered by the "interrupt" must reach the main loop through a fixed-size buffer that you write yourself.
- The interrupt side must never block, wait or allocate. If the buffer is full, count the lost bytes and carry on.

5.3 Frame parser

- Process the stream one byte at a time and recover from every kind of error in the samples.
- Count at least: good readings, good heartbeats, CRC errors, length errors, unknown frames and bytes discarded.

5.4 Power-safe flash log

- Simulate a 64 KB NOR flash with 4 KB sectors. Erasing a sector sets every byte to 0xFF. A write can only change bits from 1 to 0; your simulator must reject anything else.
- Simulate timing: a sector erase takes 20 ms; each write call takes 100 µs and may program up to 256 bytes; reads are free. Your code must wait for these operations to finish, just like a real flash driver.
- The flash contents must survive a restart of your program (for example, keep them in a file).
- Store every reading. When flash is full, discard the oldest readings and keep going.
- Provide a way to cut power at any point, including in the middle of a write or an erase. For example: a flag that stops the program after N flash bytes have been written. After a restart, every reading your firmware reported as stored must still be there, and logging must continue.

5.5 Debug console

- Read commands from standard input: `stats` (all counters, plus bytes received, bytes lost to overflow and readings in flash), `dump <n>` (the newest n stored readings) and `erase`.
- Handling a command must not cause received bytes to be lost at 115200 baud.

5.6 What "working" means

Capture	Baud	Expected result
clean.bin	115200	Every intact reading stored, zero bytes lost
noisy.bin	115200	Every intact reading stored, zero bytes lost
burst.bin	921600	Losing data is allowed, but every lost byte must be counted: bytes received + bytes lost = file size. Explain what limits you.

6. Design document (required)

Before and while you build, we want to see how you think about the whole system. Put a short design document in a `docs/` folder in your repository. One to three pages is plenty.

Any format is fine: a photo of a hand-drawn sketch, a whiteboard picture, draw.io, Excalidraw, Mermaid, PlantUML or plain text with diagrams. Neatness does not matter; clear thinking does.

Please cover:

- A block diagram of your modules and how data moves between them.
- How the interrupt side and the main loop share data, and why that is safe.
- The parser's states and how it recovers from errors.
- Your flash layout, and exactly what happens on boot after a power cut.
- The main trade-offs you made and the alternatives you rejected.

If your design changed while you were building, keep the first version and add a note on what changed and why. That is often the most interesting part for us.

7. Language and rules

Language. Use any language and toolchain you are comfortable with. Most candidates use C, C++ or Rust, but others are welcome. Whatever you choose, follow these firmware rules in the core logic:

1. No direct operating-system calls. Hardware and OS access goes only through your hardware layer.
2. No heap allocation after start-up.
3. The code that plays the interrupt never blocks or waits.

If your language makes one of these rules hard, explain in your README how you handled it.

Rules

- Building and running the tests should each take one command. Turn on your toolchain's strict warnings or lints (for example `-Wall -Wextra -Werror` or `clippy -D warnings`).
- You may use datasheets, books, documentation, man pages and normal test frameworks.
- **No AI tools**: no code assistants, chatbots or generated code. In the review meeting we will ask you to explain and change your code live, so write only what you can defend line by line.
- If you run out of time, stop and write down what you would do next. A clearly described gap is better than rushed code.
- If anything in this brief is unclear, make a reasonable assumption, note it in your README and carry on.

8. Stretch goals (optional)

Pick at most two, and only after the core is done. A finished core counts for more than any stretch goal.

- Store every intact reading from `burst.bin` at 921600 baud with zero loss.
- Run the firmware on an RTOS that runs on a normal computer, such as the FreeRTOS POSIX port, Zephyr native_sim or Embassy's std target.
- Fuzz the parser and fix what the fuzzer finds.
- Add wear levelling and report erase counts per sector.
- Add CI that builds, runs the tests and runs a static analyser.
- Cross-compile the core for an ARM Cortex-M target. It only needs to build, not run.

9. What to submit

Send a link to your git repository (public, or private with access given to us) within 48 hours.

- ☐ Source code, build files and tests. Commit as you go; we read the history, so please do not squash everything into one commit.
- ☐ `docs/`: your design document (section 6).
- ☐ `README.md`: how to build, run and test; key decisions and trade-offs; how you tested power-loss recovery; known limitations; what you would do with another day; and any assumptions you made.
- ☐ The `stats` output from a run against each of the three samples.

10. The review meeting

The meeting lasts about 60 minutes. Please have your project ready to build and run on your own machine, with screen sharing.

Part	What happens
Demo, 10 min	Run your build on a sample, and show a power cut and recovery
Your code, 20 min	We ask about decisions in your code and design document
Live change, 15 min	A small change to your code, made together
Real hardware, 10 min	How your design would move to a real microcontroller
Your questions, 5 min	Anything you want to ask us

We may also give you a new capture during the meeting and ask you to run it.

11. How we evaluate

Roughly in order of importance:

1. **Concurrency correctness:** the interrupt-to-main-loop handoff is safe, and you can explain why.
2. **Robustness:** every intact frame is found, power loss never loses stored data, and every lost byte is counted.
3. **Design:** clear modules, firmware logic separated from the simulation, and a design document that shows your thinking.
4. **Testing:** tests that cover the edge cases, not just the happy path.
5. **Code quality, README and git history:** readable code, honest trade-offs and incremental commits.

> Good luck. We look forward to seeing how you approach it.

Appendix A. Sample captures

Capture	Bytes	Intact readings	Intact heartbeats	Baud
`clean.bin`	22,400	2,000	80	115200
`noisy.bin`	36,695	2,714	110	115200
`burst.bin`	223,294	19,784	198	921600

At 115200 baud, clean.bin takes about 2 seconds to replay and noisy.bin about 3 seconds. burst.bin takes about 2.5 seconds at 921600 baud.

Appendix B. Making more test data

The samples were made with these commands. Change the seed or the profile values to create new captures; the manifest always tells you the correct counts.

Command
`python3 gen_frames.py --profile clean --seed 1001 --out clean.bin --manifest clean.json`
`python3 gen_frames.py --profile noisy --seed 2002 --out noisy.bin --manifest noisy.json`
`python3 gen_frames.py --profile burst --seed 3003 --out burst.bin --manifest burst.json`
`python3 gen_frames.py --profile noisy --seed 7 --readings 500 --p-corrupt 0.3 --out hard.bin`

Run `python3 gen_frames.py --help` for every option.